Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3

Децентралізовані застосунки

Децентралізовані системи

Виконав
аспірант групи ІТ-41ф

Новиков Д. М.

Перевірив:

д.т.н. Гаврилко Євген Володимирович

Київ 2026

Мета роботи: Розробити децентралізовану систему відповідно до індивідуального завдання.

Завдання роботи:

Відповідно до індивідуальних завдань до лабораторної роботи №1, розробіть децентралізовану систему. Дозволяється використовувати сервер лише як реєстр вузлів.

Для підтвердження цілісності даних або автентичності відправника використовуйте хешування або цифровий підпис.

Сервер використовується лише як реєстр вузлів, а взаємодія відбувається напряму між вузлами.

Варіант 4:

4. Захищена (шифрування XOR) передача повідомлень по мережі.

Хід роботи:

Використані технології:

- Платформа: .NET 9 (C#);
- Протоколи комунікації: gRPC (HTTP/2) для високопродуктивної міжвузлової взаємодії (P2P) та зв'язку з трекером; REST API (HTTP/1.1) зі Swagger UI для взаємодії з користувачем;
- Криптографія: Гібридне шифрування (швидкий симетричний шифр XOR для тексту повідомлень + надійний асиметричний RSA-2048 для обміну ключами), гешування SHA-256 для створення цифрових підписів;
- Інфраструктура: Docker та Docker Compose.

Архітектура Системи:

Система побудована за моделлю гібридної однорангової мережі (Hybrid P2P) з використанням принципів чистої архітектури та децентралізації. Вона складається з двох ключових типів компонентів:

- Registry Server (Tracker): Централізований gRPC-сервер, який виконує роль "телефонної книги". Він зберігає список активних вузлів (Node ID, IP-адреса, публічний RSA-ключ) у потокобезпечній пам'яті. Трекер не бере участі в маршрутизації самих повідомлень, забезпечуючи тим самим децентралізацію трафіку;
- Peer Node (Гібридний вузол): Кожен учасник мережі є незалежним мікросервісом, який одночасно виступає і сервером, і клієнтом:

- Server (gRPC Listener): Фоновий процес, що приймає прямі вхідні з'єднання від інших вузлів. Він перевіряє цифрові підписи, розшифровує сесійні ключі та повідомлення, після чого зберігає їх у локальне in-memory сховище (MessageStore);
- Client (REST API): Веб-інтерфейс, через який користувач ініціює відправку. Сервіс автоматично отримує публічний ключ цільового вузла з Registry, генерує унікальний XOR-ключ для кожного повідомлення, виконує багаторівневе шифрування та відправляє payload напряму цільовому вузлу;
- Security & Identity Module: Окремий сервіс (NodeIdentityManager), який відповідає за автоматичну генерацію унікальної пари RSA-ключів при старті вузла та забезпечує конфіденційність, цілісність і автентичність даних (Authentication & Integrity);
- Lifecycle & Registration Module: Фоновий сервіс (NodeInitializationService, реалізований на базі IHostedService), який керує життєвим циклом вузла в мережі. Під час запуску додатку він автоматично реєструє присутність вузла на центральному трекері (передаючи свій NodeId, gRPC-адресу для вхідних з'єднань та публічний ключ), а при зупинці контейнера – виконує коректне відключення, забезпечуючи актуальність глобального списку активних учасників.

Алгоритм шифрування та забезпечення цілісності даних:

У системі реалізовано гібридний криптографічний протокол, який поєднує високу швидкодію симетричного шифрування з безпекою розподілу ключів асиметричної криптографії, а також використовує механізм електронного цифрового підпису (ЕЦП) для гарантування цілісності та автентичності.

1. Етап ініціалізації вузла (Генерація довгострокових ключів):
   При старті кожен вузол (Peer Node) автоматично генерує унікальну пару асиметричних ключів за алгоритмом RSA з довжиною ключа 2048 біт.
   a. Приватний ключ (Private Key): Зберігається виключно в оперативній пам'яті поточного вузла;
   b. Публічний ключ (Public Key): Експортується у форматі SubjectPublicKeyInfo (SPKI), кодується у Base64 та відправляється на центральний трекер для вільного доступу іншими учасниками мережі.

2. Процес відправлення повідомлення (Відправник A -> Одержувач B):
   Коли вузол A ініціює відправку повідомлення вузлу B, алгоритм виконує наступні кроки:
   a. Отримання публічного ключа: Вузол A запитує з трекера Публічний RSA-ключ Одержувача B;
   b. Генерація сесійного ключа: Для кожного нового повідомлення генерується унікальний випадковий 32-байтний (256-бітний) симетричний ключ за допомогою криптографічно стійкого генератора (RandomNumberGenerator);
   c. Шифрування тексту (Конфіденційність): Відкритий текст (P) шифрується симетричним алгоритмом XOR з використанням згенерованого сесійного ключа (K). Результатом є зашифрований масив байтів (EncryptedContent);
   d. Електронний цифровий підпис (Цілісність та Автентичність): Відправник A створює криптографічний геш (SHA-256) від оригінального відкритого тексту повідомлення і підписує цей геш своїм власним Приватним RSA-ключем (з використанням стандарту паддінгу RSASignaturePadding.Pkcs1). Результатом є DigitalSignature;
   e. Інкапсуляція сесійного ключа (Безпечна передача): Згенерований 32-байтний XOR-ключ шифрується Публічним RSA-ключем Одержувача B. Результатом є EncryptedXorKey.

   Після цього по P2P-мережі передається структура (DirectMessage), яка містить лише зашифровані компоненти: EncryptedContent, EncryptedXorKey та DigitalSignature.
3. Процес отримання повідомлення (Одержувач B):
   При отриманні вхідного запиту алгоритм працює у зворотному порядку, застосовуючи принцип "Zero Trust" (нульової довіри) до вхідних даних:
   a. Розшифрування сесійного ключа: Одержувач B використовує свій Приватний RSA-ключ для розшифрування EncryptedXorKey. Завдяки властивостям RSA, ніхто інший у мережі (навіть трекер) не може розшифрувати цей ключ;
   b. Розшифрування тексту: Отриманий сесійний ключ (K) застосовується до зашифрованого тексту (EncryptedContent). Завдяки природі операції XOR, алгоритм дешифрування ідентичний алгоритму шифрування. Одержувач отримує відкритий текст повідомлення (P);

c. Верифікація підпису: Одержувач B завантажує Публічний ключ Відправника A з трекера. Використовуючи цей відкритий ключ, алгоритм перевіряє валідність DigitalSignature відносно розшифрованого відкритого тексту;
d. Валідація: Якщо математична перевірка підпису успішна – система гарантує, що повідомлення було відправлено саме вузлом A і не було модифіковано (Man-in-the-Middle) під час транзиту. Якщо перевірка дає збій – генерується виняток безпеки (Security Alert), і повідомлення відхиляється.

Інфраструктура та Розгортання (Docker):

Використання Docker Compose дозволяє розгорнути центральний трекер та будь-яку кількість незалежних P2P-вузлів (наприклад, Alice, Bob, Nick) однією командою, симулюючи реальну мережу на локальній машині.

- Network Isolation & Internal DNS: Усі компоненти взаємодіють всередині закритої віртуальної мережі Docker (bridge network). Вузли знаходять один одного за внутрішніми іменами контейнерів (наприклад, peer-alice:5001), минаючи зовнішню мережу хост-машини;
- Dual-Protocol Binding (Port Mapping): Веб-сервер Kestrel на кожному вузлі налаштований на паралельне прослуховування двох портів. gRPC-порти (5000-ті) повністю ізольовані всередині Docker-мережі для безпечного P2P-зв'язку, тоді як REST-порти (8081, 8082, 8083) прокинуті на хост-машину для доступу до Swagger UI.

Приклад використання:

Запустимо наш docker-compose сценарій, який містить 1 центральний трекер (Registry Server) та 3 незалежні P2P-вузли (Alice, Bob, Nick). Кожен вузол одночасно виконує роль gRPC-сервера (для прийому повідомлень) та REST API-шлюзу (для взаємодії з користувачем через Swagger):

docker-compose:

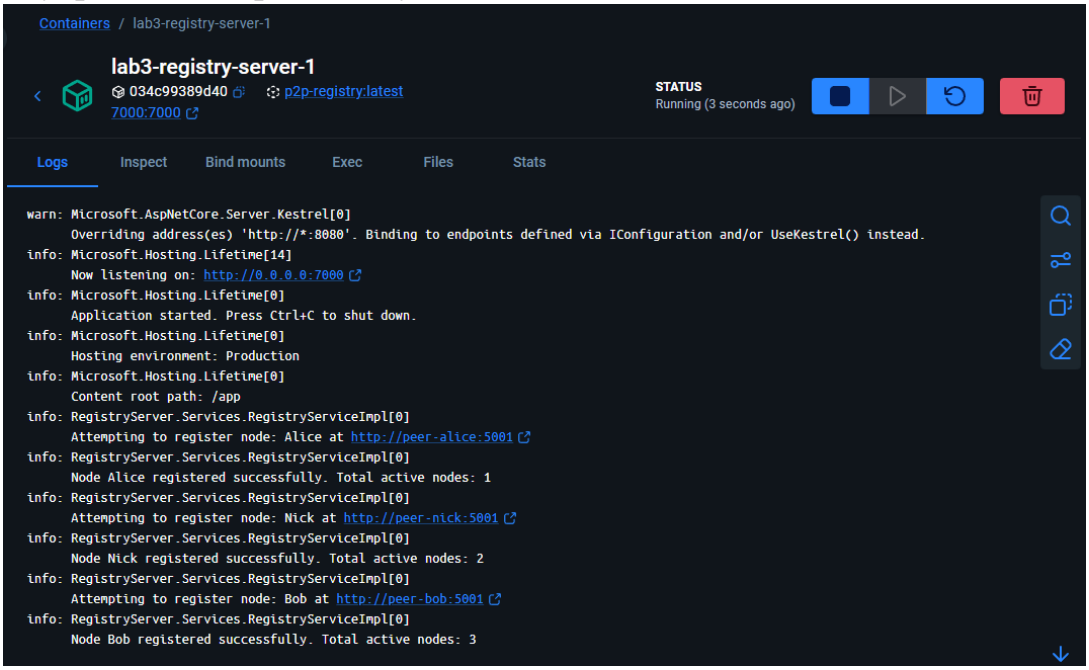
```
services:
  registry-server:
    image: p2p-registry
    build:
      context: .
      dockerfile: DecentralizedChat.RegistryServer/Dockerfile
    ports:
      - "7000:7000"
    networks:
      - p2p-network
```

```yaml
# First peer client - Alice.
peer-alice:
  image: p2p-node
  build:
    context: .
    dockerfile: DecentralizedChat.PeerNode/Dockerfile
  ports:
    - "8081:8081" # Map Swagger to localhost:8081
  environment:
    - ASPNETCORE_ENVIRONMENT=Development
    - ASPNETCORE_URLS=http://+:8081;http://+:5001
    - Logging__LogLevel__Default=Debug
    - NodeConfig__NodeId=Alice
    - NodeConfig__ListenAddress=http://peer-alice:5001
    - NodeConfig__RegistryAddress=http://registry-server:7000
  networks:
    - p2p-network
  depends_on:
    - registry-server

# Second peer client - Bob.
peer-bob:
  image: p2p-node
  build:
    context: .
    dockerfile: DecentralizedChat.PeerNode/Dockerfile
  ports:
    - "8082:8081" # Map Swagger to localhost:8082
  environment:
    - ASPNETCORE_ENVIRONMENT=Development
    - ASPNETCORE_URLS=http://+:8081;http://+:5001
    - Logging__LogLevel__Default=Debug
    - NodeConfig__NodeId=Bob
    - NodeConfig__ListenAddress=http://peer-bob:5001
    - NodeConfig__RegistryAddress=http://registry-server:7000
  networks:
    - p2p-network
  depends_on:
    - registry-server

# Third peer client - Nick.
peer-nick:
  image: p2p-node
  build:
    context: .
    dockerfile: DecentralizedChat.PeerNode/Dockerfile
  ports:
```

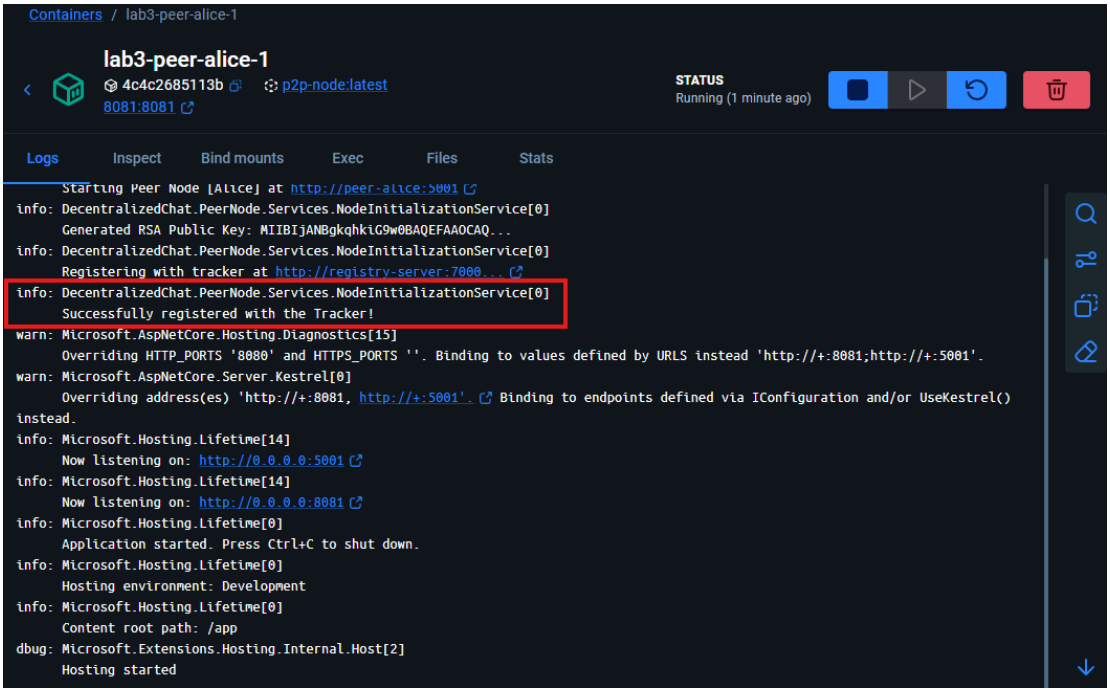
```yaml
      - "8083:8081" # Map Swagger to localhost:8083
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=http://+:8081;http://+:5001
      - Logging__LogLevel__Default=Debug
      - NodeConfig__NodeId=Nick
      - NodeConfig__ListenAddress=http://peer-nick:5001
      - NodeConfig__RegistryAddress=http://registry-server:7000
    networks:
      - p2p-network
    depends_on:
      - registry-server

networks:
  p2p-network:
    driver: bridge
```

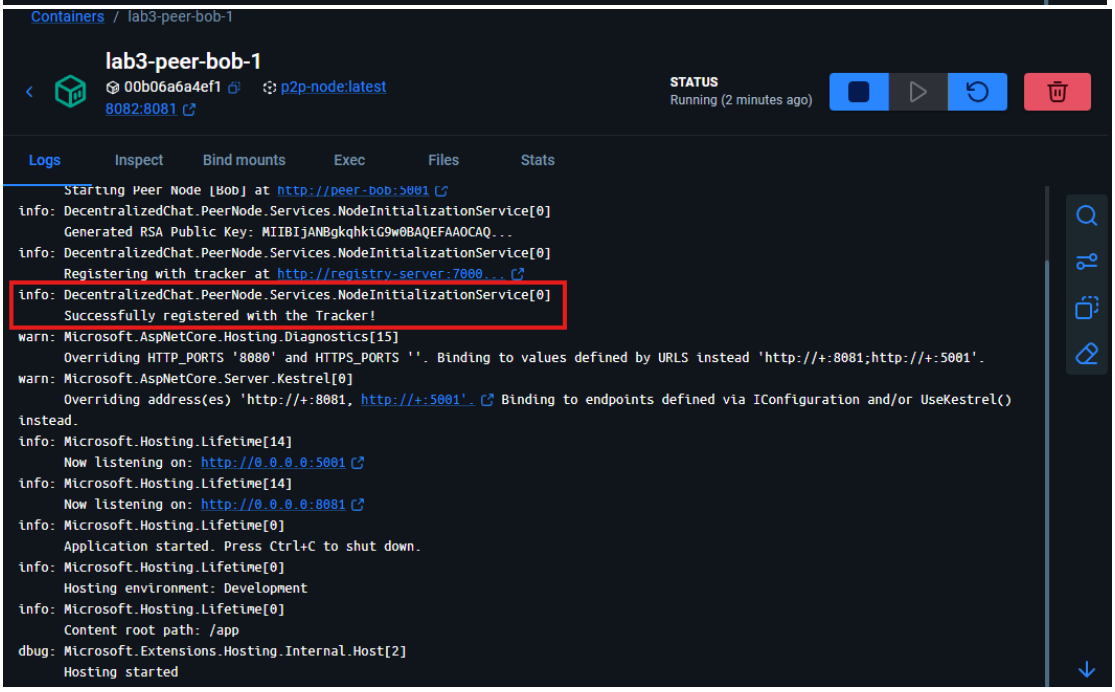
Перевіримо логи наших компонентів під час запуску:

- Registry Server: Як бачимо з логів сервера, трекер успішно запустився і зареєстрував три незалежні вузли, зберігши їхні внутрішні IP-адреси та публічні RSA-ключі:



- Peer Nodes (Alice, Bob, Charlie): Кожен вузол успішно згенерував свою унікальну пару RSA-ключів, виконав процес реєстрації (Registration) на трекері та почав прослуховувати внутрішній порт 5001 для вхідних P2P-з'єднань, а також зовнішні порти (8081, 8082, 8083) для Swagger UI:

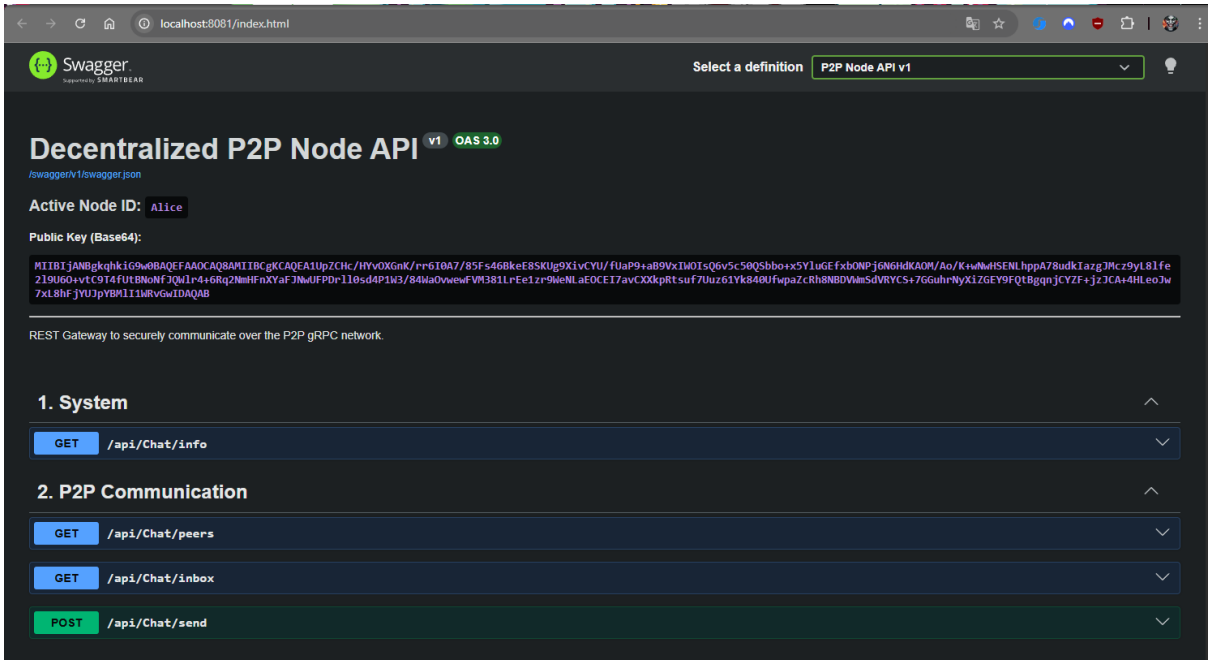




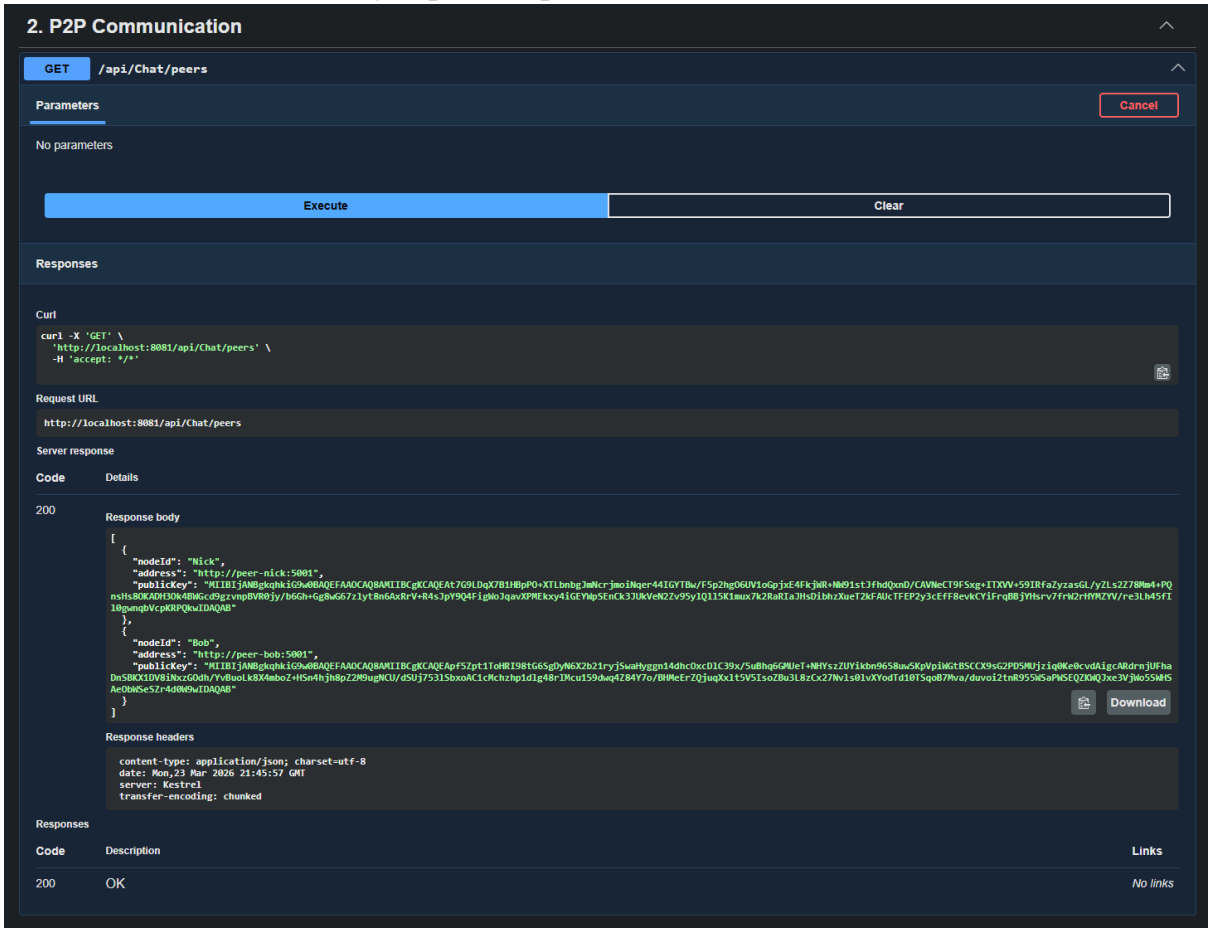
Сценарій взаємодії (Публікація та обробка даних):

Крок 1: Перевірка активних учасників мережі (Node Discovery)

Відкриваємо веб-браузер з інтерфейсом Swagger для вузла Alice (http://localhost:8081). Завдяки динамічному Swagger-фільтру, у заголовку сторінки ми бачимо ідентифікатор поточного вузла (Active Node ID: Alice) та його публічний ключ:



Щоб дізнатися, кому можна відправити повідомлення, виконуємо GET-запит до ендпоінту /api/chat/peers:



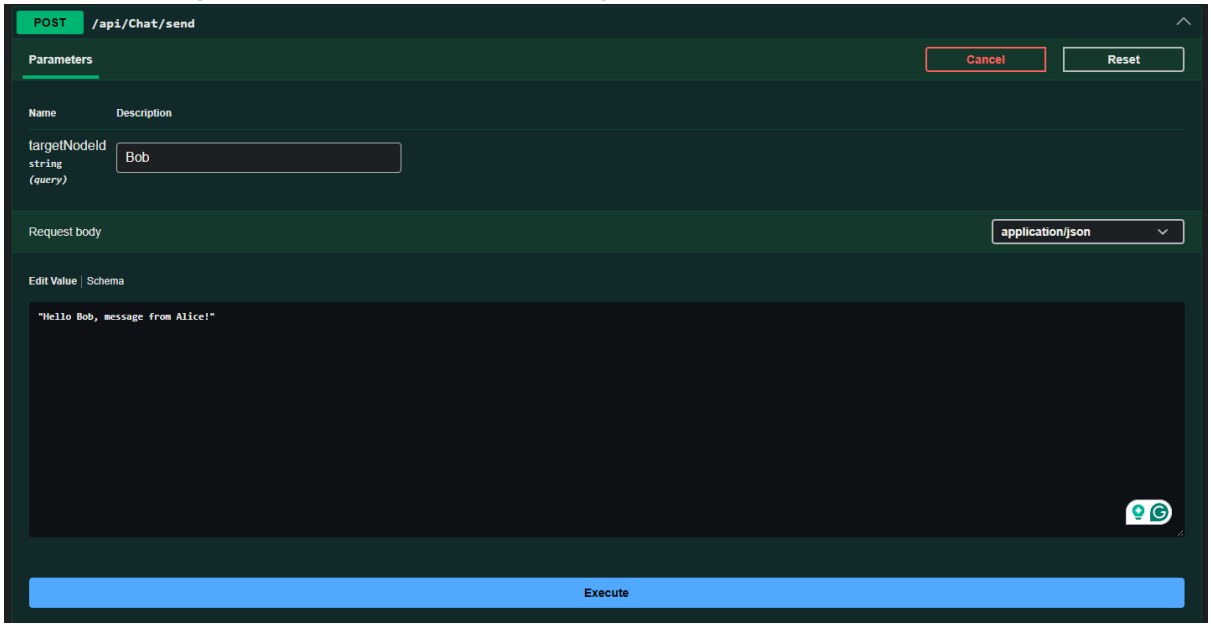
У тілі відповіді ми отримуємо масив доступних вузлів (Bob та Nick) з їхніми внутрішніми Docker-адресами та Base64-кодованими публічними ключами.

Крок 2: Відправка зашифрованого повідомлення (Alice -> Bob)

В інтерфейсі Alice переходимо до POST-запиту /api/chat/send. У параметрах вказуємо:

targetNodeId: Bob

messageText: "Hello Bob, message from Alice!"

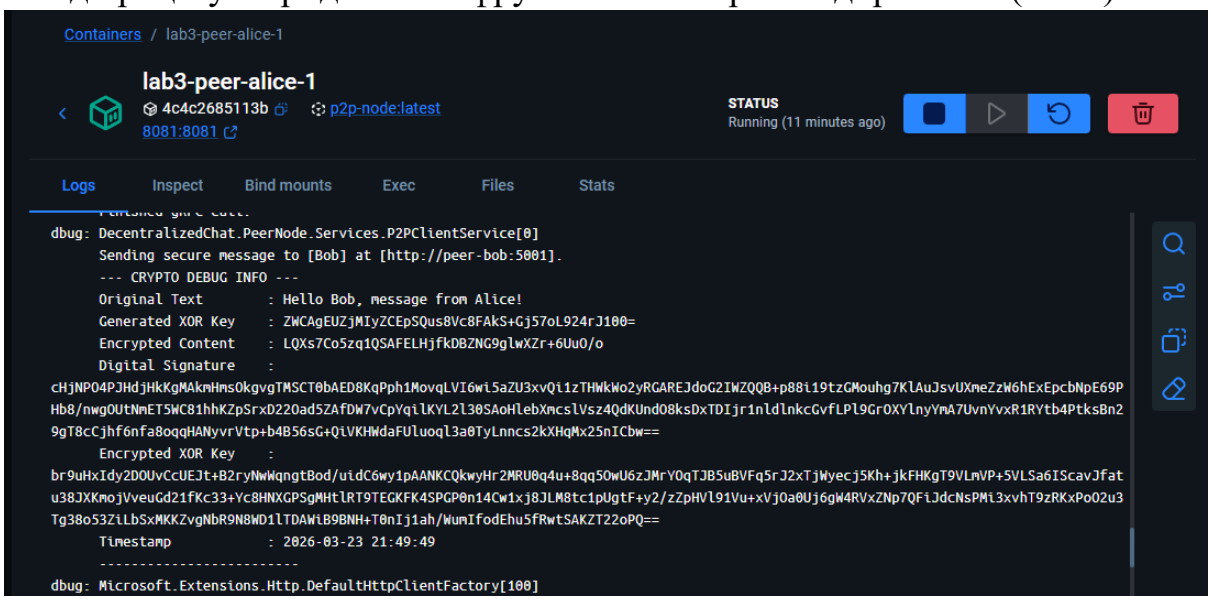


Натискаємо Execute. Вузол Alice виконує наступне:

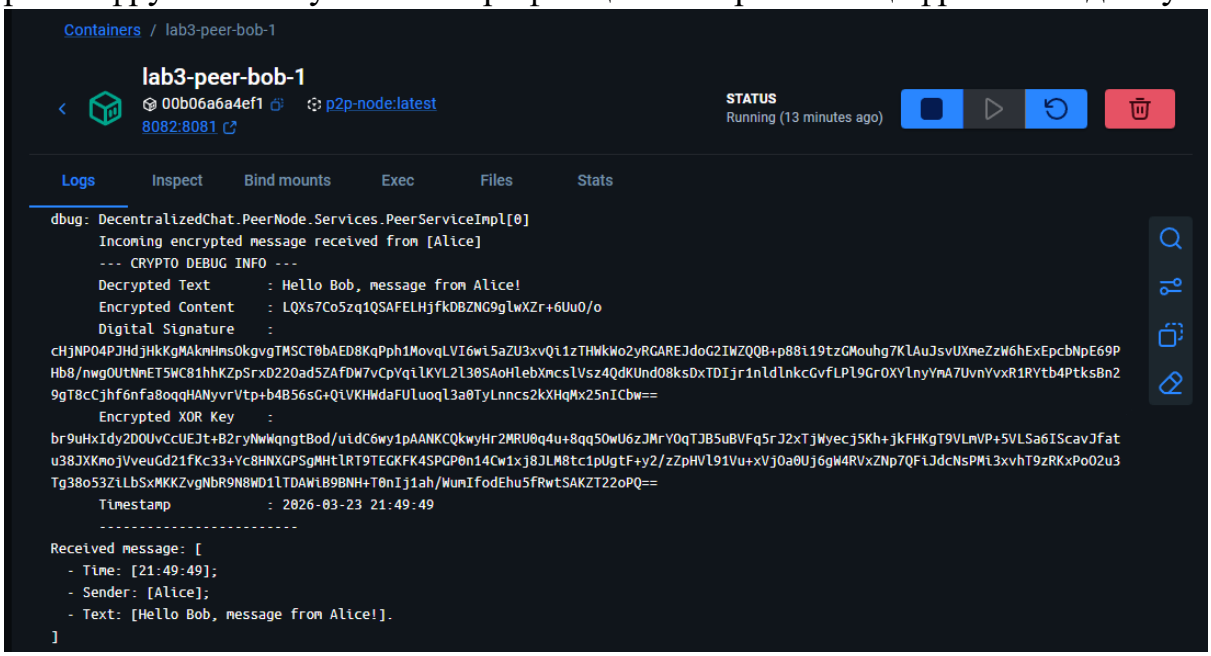
1. Запитує у трекера публічний ключ Bob'a;
2. Генерує випадковий 32-байтний XOR-ключ;
3. Шифрує текст повідомлення XOR-ключем;
4. Підписує оригінальний текст своїм приватним RSA-ключем (SHA-256);
5. Шифрує XOR-ключ публічним RSA-ключем Bob'a;
6. Встановлює пряме gRPC-з'єднання з контейнером Bob'a (http://peer-bob:5001) та передає зашифрований payload.

Крок 3: Аналіз криптографічних логів (LogLevel == Debug)

Перевіривши консоль Docker Compose, ми бачимо деталізований вивід процесу гібридного шифрування на стороні відправника (Alice):

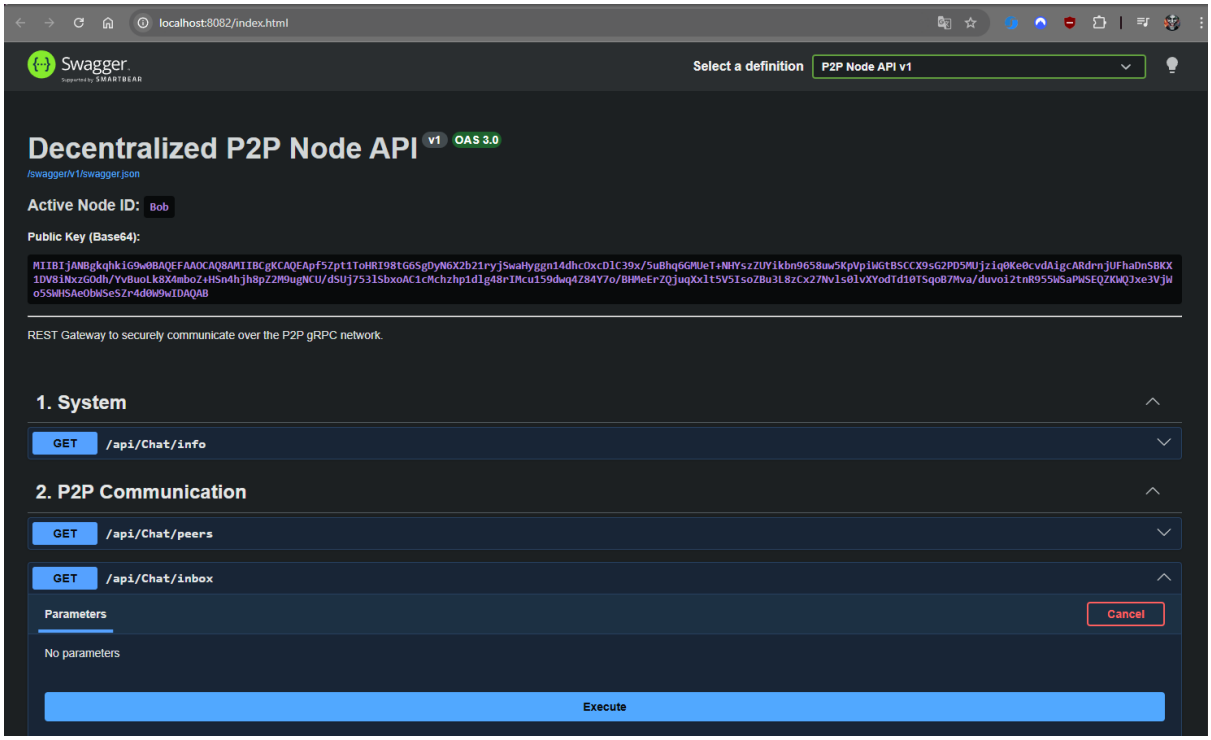


На стороні отримувача (Bob) в логах фіксується дзеркальний процес розшифрування та успішна верифікація електронного цифрового підпису:



Крок 4: Отримання та розшифрування повідомлення (Bob)

Відкриваємо сторінку Swagger для вузла Bob (http://localhost:8082). Виконуємо GET-запит до ендпоінту /api/chat/inbox. Оскільки фоновий gRPC-сервіс Bob'a вже перевірив підпис і розшифрував дані своїм приватним RSA-ключем, у тілі відповіді ми бачимо структурований JSON (DTO) із розшифрованим текстом:



```
[
  {
    "senderId": "Alice",
    "timestamp": "2026-03-23T21:49:49+00:00",
    "messageText": "Hello Bob, message from Alice!"
  }
]
```

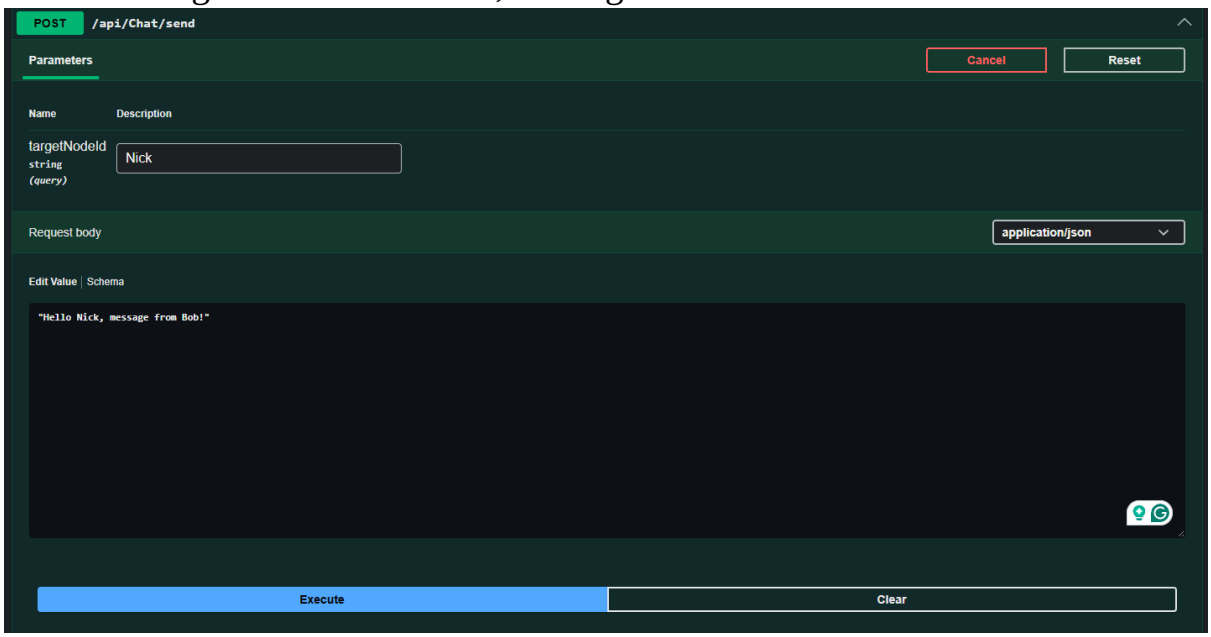

Повідомлення успішно отримано та збережено в локальній пам’яті вузла Bob.

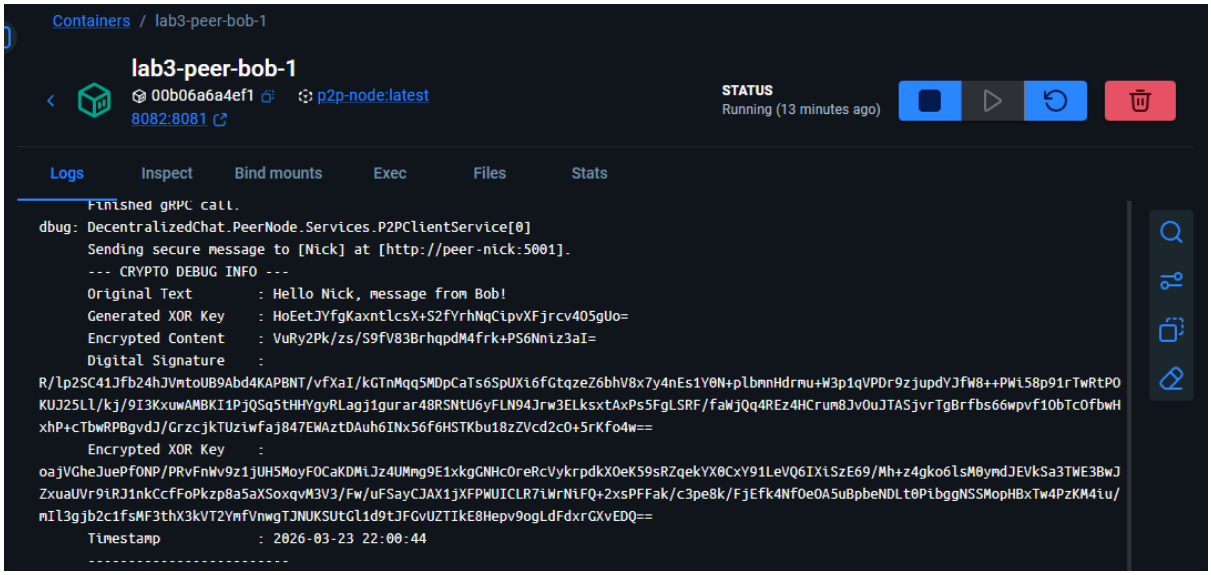
Крок 5: Децентралізована маршрутизація (Bob -> Nick)

Щоб продемонструвати справжню однорангову (P2P) природу системи, в інтерфейсі Bob'a відправляємо повідомлення до третього вузла – Nick:

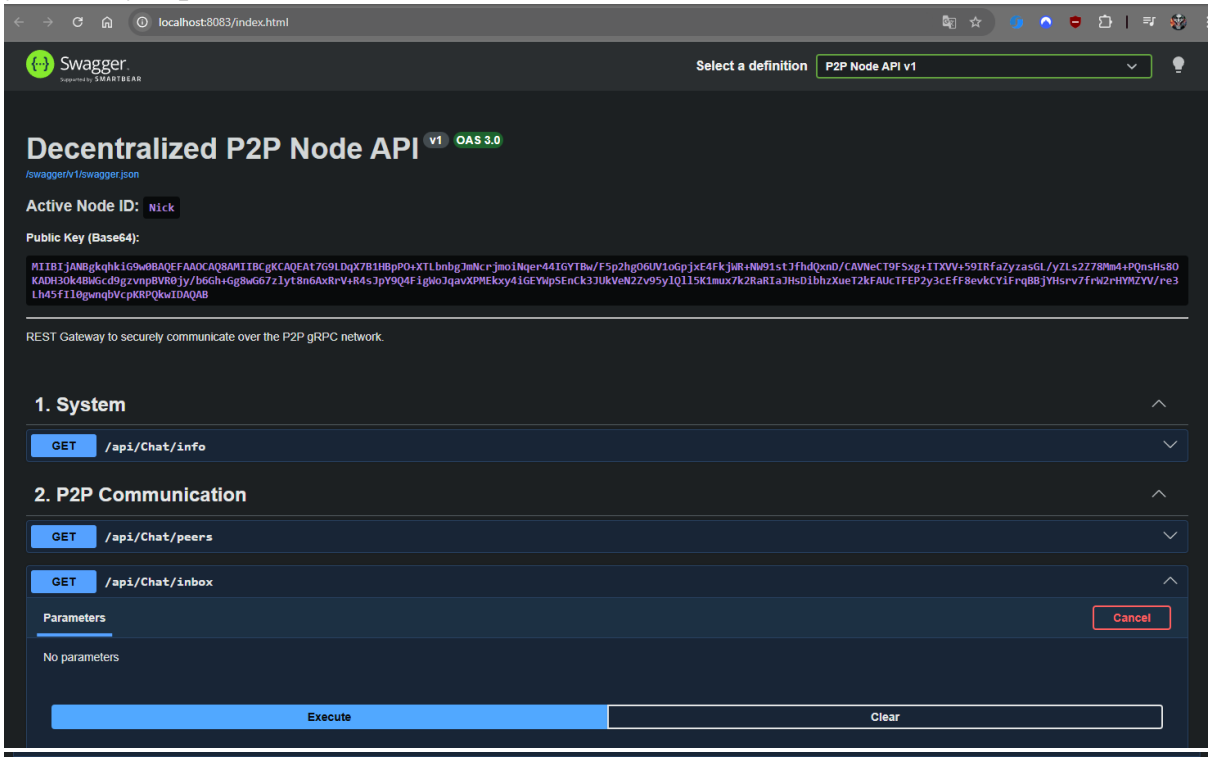
targetNodeId: Nick

messageText: "Hello Nick, message from Bob!"

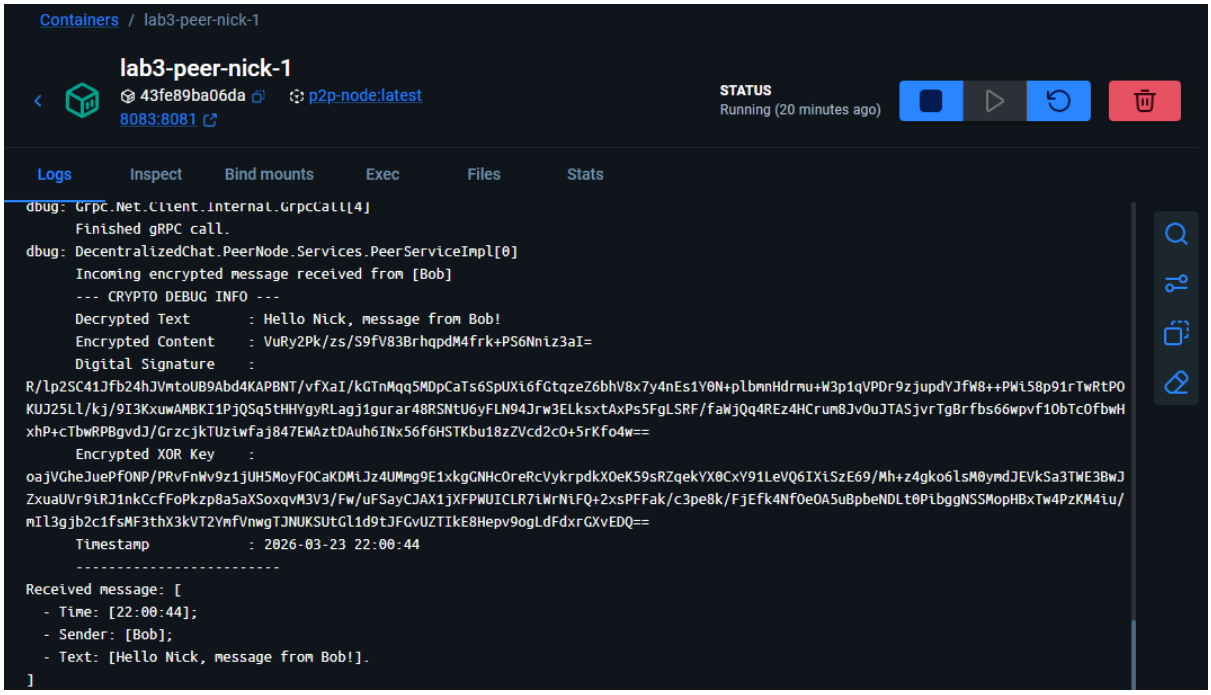




Повідомлення успішно доставляється. Центральний трекер та вузол Alice не беруть жодної участі в транзиті цього повідомлення і не мають змоги його перехопити. Вузол Nick успішно розшифровує його і відображає у своєму /api/chat/inbox:



Так само, перевіримо логи docker:

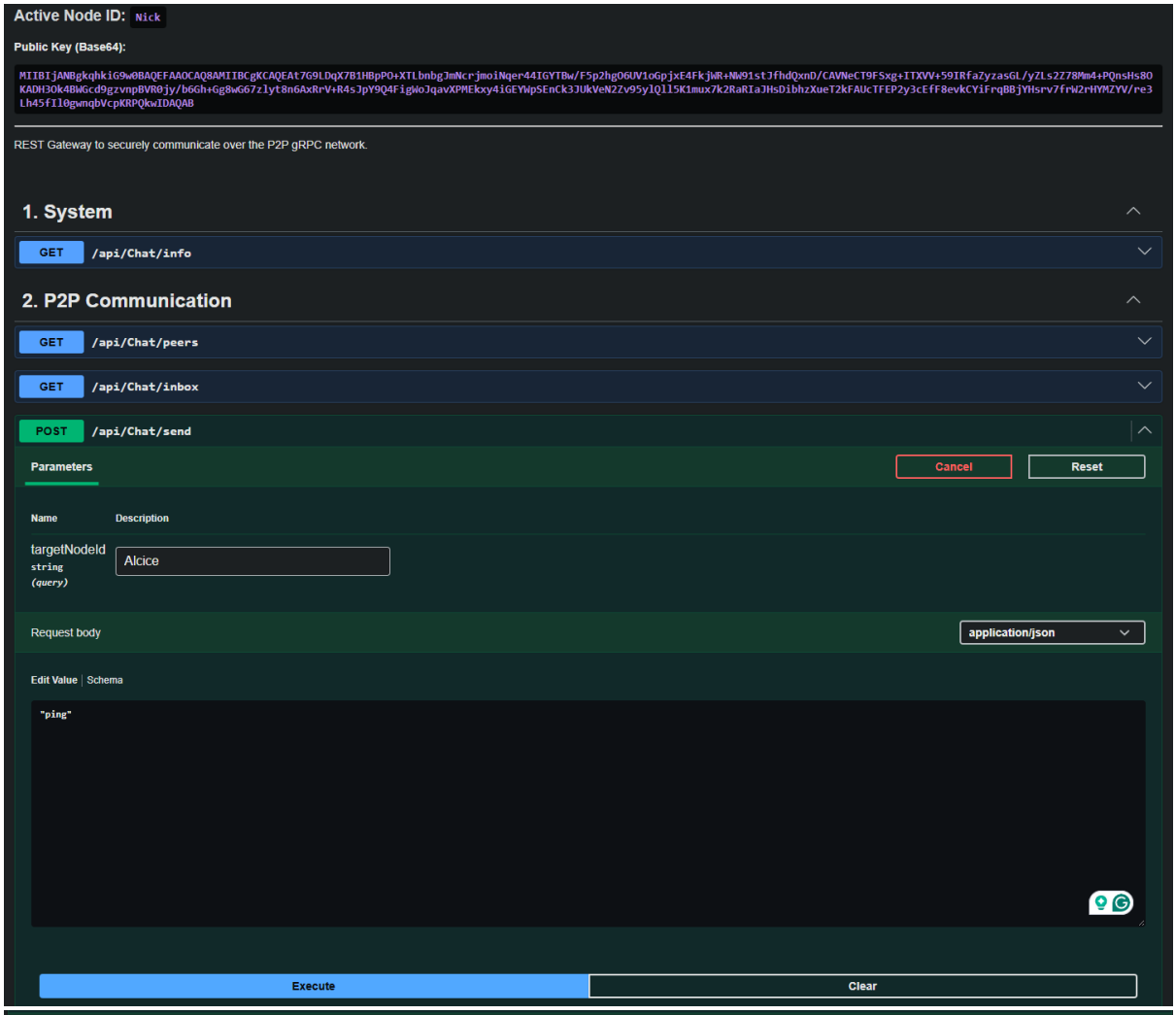


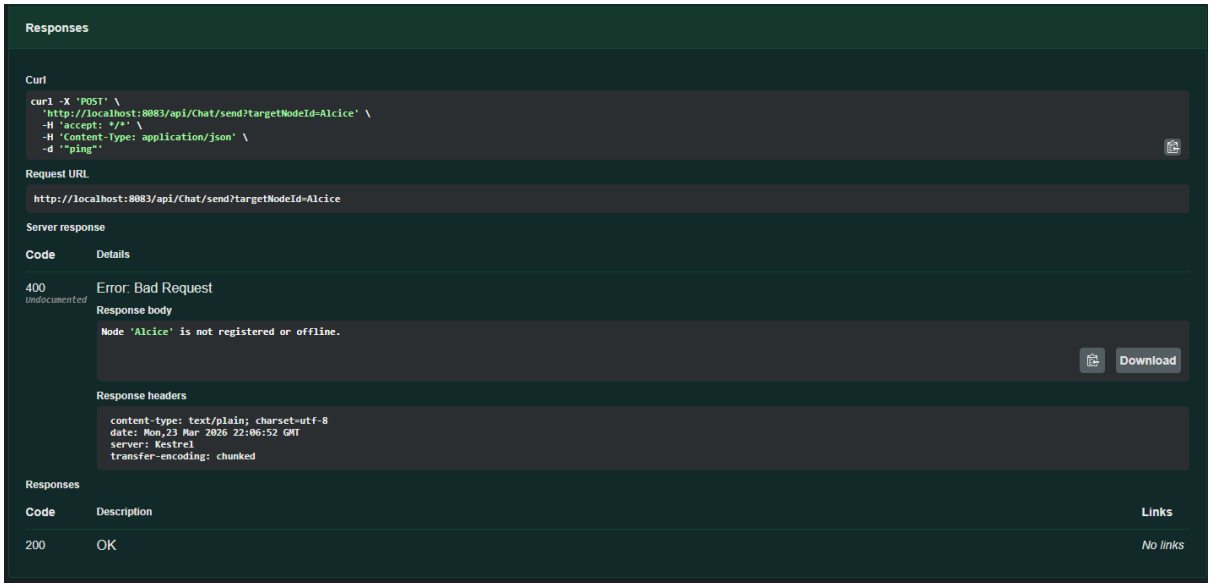
Крок 6: Перевірка механізму Node Discovery (Nick -> Alcice – неіснуючий вузол)

Спробуємо направити повідомлення вузлу, якого не існує в реєстрі нашого Tracker (Registry Server). Для цього з вузла Nick спробуємо надіслати повідомлення неіснуючому адресату Alcice (з навмисною помилкою в імені):

targetNodeId: Alcice

messageText: "ping"





У відповідь від REST API отримуємо повідомлення про помилку (400 Bad Request): Node 'Alcice' is not registered or offline.

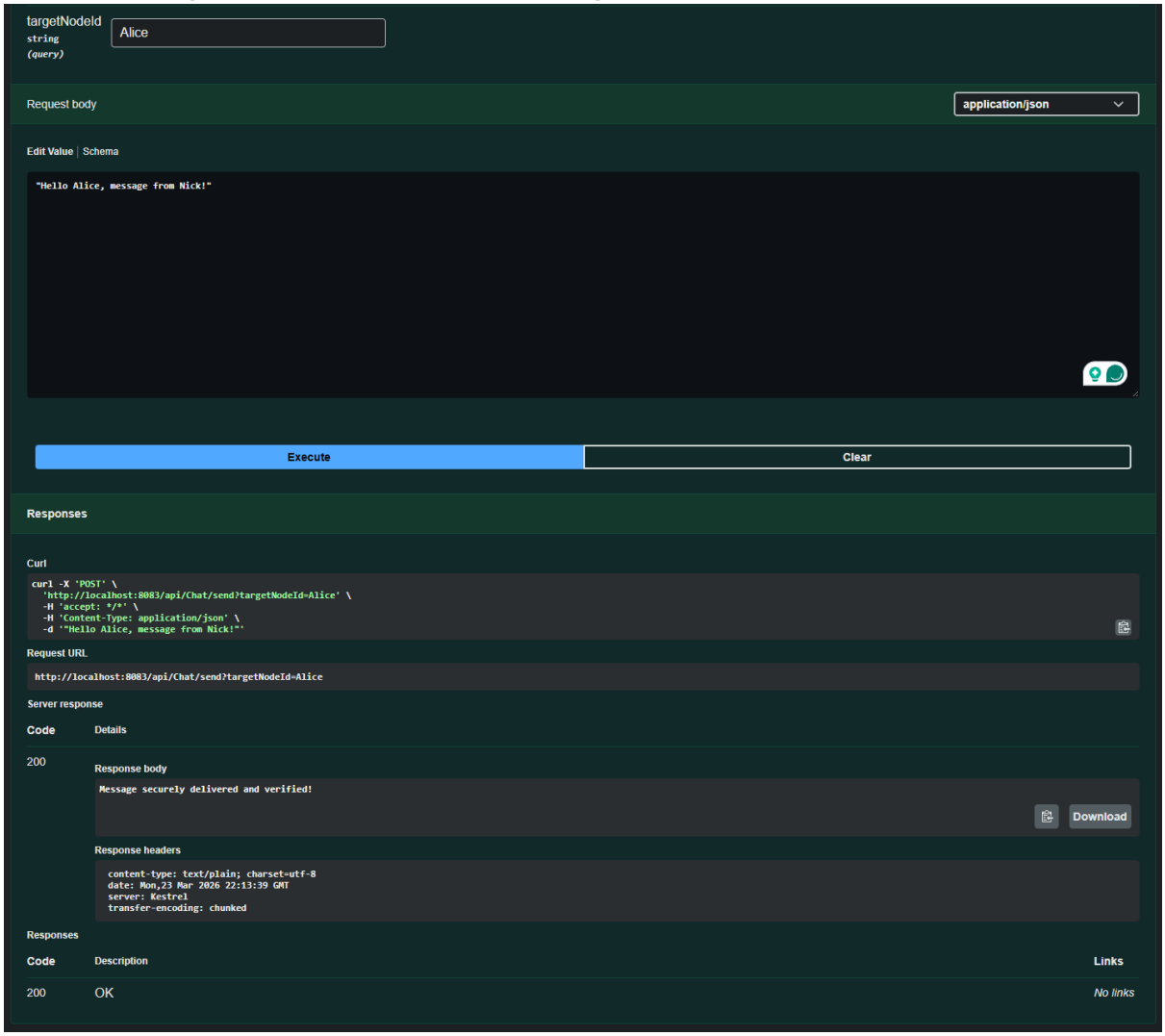
Це підтверджує використання сервісу Registry як єдиного джерела правди та надійного запобіжника, який блокує спроби встановлення P2P-з'єднання з невідомими або неактивними хостами ще на етапі валідації.

Крок 7: Фінальна перевірка маршрутизації (Nick -> Alice)

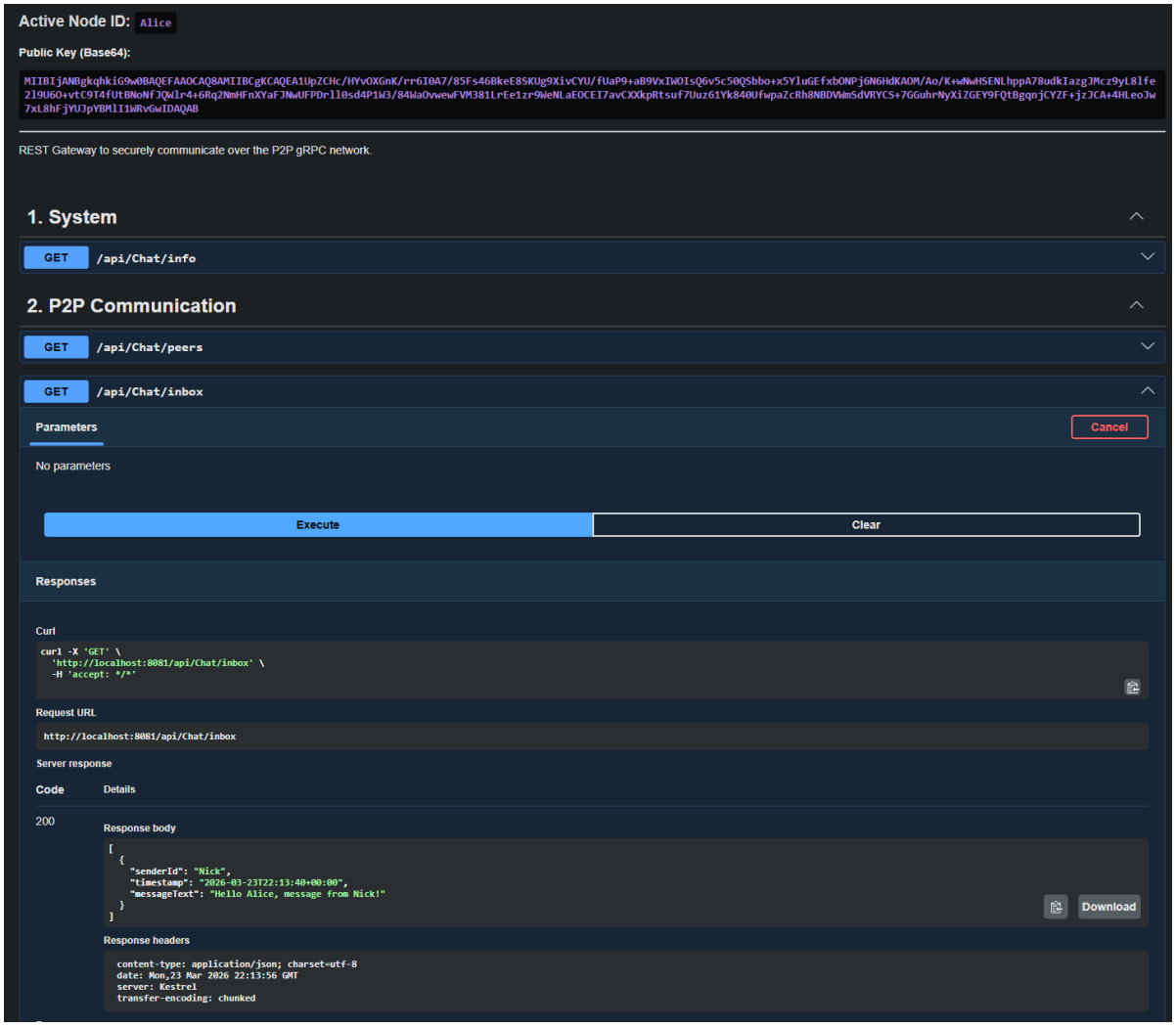
Для фінальної перевірки направимо валідне повідомлення з вузла Nick до Alice:

targetNodeId: Alice

messageText: "Hello Alice, message from Nick!"



Повідомлення було успішно опрацьовано, зашифровано та надіслано вузлом Nick, а потім отримано й коректно розшифровано вузлом Alice (що підтверджується GET-запитом до /api/chat/inbox на стороні Alice):



Цей сценарій повністю підтверджує успішну реалізацію децентралізованої системи обміну зашифрованими повідомленнями з використанням мікросервісної архітектури та Docker.

Висновки: за результатом виконання лабораторної роботи спроєктовано та реалізовано розподілену однорангову (Peer-to-Peer) мікросервісну систему на базі гібридної архітектури. Головною метою було створення безпечного, децентралізованого середовища для прямого обміну даними між незалежними вузлами з гарантією конфіденційності, цілісності та автентичності повідомлень. Цю мету було досягнуто завдяки розробці централізованого сервісу виявлення вузлів (Registry Server / Node Discovery) та гібридних клієнт-серверних додатків (Peer Nodes), які взаємодіють через високопродуктивний протокол gRPC (HTTP/2). На стороні вузлів реалізовано паралельну роботу REST API шлюзу (зі Swagger UI) та фонових gRPC-слухачів, а також впроваджено криптографічний протокол гібридного шифрування (поєднання RSA-2048, симетричного XOR-шифру та цифрових підписів SHA-256). Крім того, забезпечено повну

оркестрацію, маршрутизацію та мережеву ізоляцію всіх незалежних компонентів у віртуальній мережі контейнеризованого середовища Docker Compose.

Вихідний код застосунку можна знайти за наступним посиланням на GitHub.